

Colombian Collegiate Programming League

CCPL 2026

Round 10 -- September 26

Problems

This set contains 11 problems; pages 1 to 14.

(Borrowed from several sources online.)

A - Automatic Answer	1
B - Anti-brute Force Lock	2
C - Collecting Beepers	4
D - Lucky Thief	5
E - Frequent Values	6
F - Frosh Week	7
G - Magic Numbers	8
H - Scrolling Sign	9
I - Integer Sequences from Addition of Terms	10
J - The Mailbox Manufacturers Problem	12
K - Asteroid Rangers	14

Official site <http://programmingleague.org>

Follow us on X: @CCPL2003

A - Automatic Answer

Source file name: answer.c, answer.cpp, answer.java, or answer.py

Last month Alice nonchalantly entered her name in a draw for a Tapmaster 4000. Upon checking her mail today, she found a letter that read:

“Congratulations, Alice! You have won a Tapmaster 4000. To claim your prize, you must answer the following skill testing question.”

Alice’s initial feelings of surprised joy turned quickly to those of dismay. Her lifetime record for skill testing questions is an abysmal 3 right and 42 wrong.

Mad Skills, the leading skill testing question development company, was hired to provide skill testing questions for this particular Tapmaster 4000 draw. They decided to create a different skill testing question to each winner so that the winners could not collaborate to answer the question.

Can you help Alice win the Tapmaster 4000 by solving the skill testing question?

Input

The input begins with t ($0 \leq t$), the number of test cases. Each test case contains an integer n ($-1000 \leq n \leq 1000$) on a line by itself. This n should be substituted into the skill testing question below.

The input must be read from standard input.

Output

For each test case, output the answer to the following skill testing question on a line by itself:

“Multiply n by 567, then divide the result by 9, then add 7492, then multiply by 235, then divide by 47, then subtract 498. What is the digit in the tens column?”

The output must be written to standard output.

Sample Input	Sample Output
2	1
637	3
-120	

B - Anti-brute Force Lock

Source file name: `brute.c`, `brute.cpp`, `brute.java`, or `brute.py`

Lately, there is one serious problem with Panda Land Safe Box: several safes have been robbed! The safes are using old 4-digits rolling lock combination (you only have to roll the digit, either up or down, until all four of them match the key). Each digit is designed to roll from 0 to 9. Rolling-up at 9 will make the digit become 0, and rolling-down at 0 will make the digit become 9. Since there are only 10000 possible keys, 0000 through 9999, anyone can try all the combinations until the safe is unlocked.



What's done is done. But in order to slow down future robbers' attack, Panda Security Agency (PSA) has devised a new safer lock with multiple keys. Instead of using only one key combination as the key, the lock now can have up to N keys which has to be all unlocked before the safe can be opened. These locks works as the following:

- Initially the digits are at 0000.
- Keys can be unlocked in any order, by setting the digits in the lock to match the desired key and then pressing the UNLOCK button.
- A magic JUMP button can turn the digits into any of the unlocked keys without doing any rolling.
- The safe will be unlocked if and only if all the keys are unlocked in a minimum total amount of rolling, excluding JUMP (yes, this feature is the coolest one).
- If the number of rolling is exceeded, then the digits will be reset to 0000 and all the keys will be locked again. In other word, the state of the lock will be reset the cracking is failed.

For example, if the keys to be unlocked are 1111, 1155, and 5511, then this can be done in 20 turns:

- Turn 0000 into 1111, rolls: 4.
- Turn 1111 into 1155, rolls: 8.
- Jump 1155 into 1111, we can do this because 1111 has been unlocked before.
- Turn 1111 into 5511, rolls: 8.

PSA is quite confident that this new system will slow down the cracking, giving them enough time to identify and catch the robbers. In order to determine the minimum number of rolling needed, PSA wants you to write a program. Given all the keys, calculate the minimum number of rolls needed to unlock the safe.

Input

The first line of input contains an integer T , the number of test cases follow. Each case begins with an integer N ($1 \leq N \leq 500$), the number of keys. Then next N four digits numbers (leading zero allowed) follow representing the keys to be unlocked. Numbers are separated by blanks.

The input must be read from standard input.

Output

For each case, print in a single line the minimum number of rolls needed to unlock all the keys.

The output must be written to standard output.

Sample Input	Sample Output
4	16
2 1155 2211	20
3 1111 1155 5511	26
3 1234 5678 9090	17
4 2145 0213 9113 8113	

C - Collecting Beepers

Source file name: collect.c, collect.cpp, collect.java, or collect.py

Karel is a robot who lives in a rectangular coordinate system where each place is designated by a set of integer coordinates (x and y). Your job is to design a program that will help Karel pick up a number of beepers that are placed in her world. To do so you must direct Karel to the position where each beeper is located. Your job is to write a computer program that finds the length of the shortest path that will get Karel from her starting position, to each of the beepers, and return back again to the starting position. Karel can only move along the x and y axis, never diagonally.

Moving from one position (i, j) to an adjacent position $(i, j + 1)$, $(i, j - 1)$, $(i - 1, j)$, or $(i + 1, j)$ has a cost of 1 unit.

You can assume that Karel's world is never larger than 20×20 squares and that there will never be more than 10 beepers to pick up. Each coordinate will be given as a pair (x, y) where each value will be in the range 1 through the size of that particular direction of the coordinate system.

Input

First there will be a line containing the number of scenarios you are asked to help Karel in. For each scenario there will first be a line containing the size of the world. This will be given as two integers (x -size and y -size). Next there will be one line containing two numbers giving the starting position of Karel. On the next line there will be one number giving the number of beepers. For each beeper there will be a line containing two numbers giving the coordinates of each beeper.

The input must be read from standard input.

Output

The output will be one line per scenario, giving the minimum distance that Karel has to move to get from her starting position to each of the beepers and back again to the starting position.

The output must be written to standard output.

Sample Input	Sample Output
<pre> 1 10 10 1 1 4 2 3 5 5 9 4 6 5 </pre>	<pre> The shortest path has length 24 </pre>

D - Lucky Thief

Source file name: lucky.c, lucky.cpp, lucky.java, or lucky.py

A very lucky thief found n keys on a street with m houses. He knows that each key opens exactly one door so he wants to know which door is, but he also wants to do the less number of tries in order to avoid the security systems.

Input

The first line contains a single integer T ($T \geq 0$) indicating the number of test cases. The following T lines contain two integers n and m , $1 \leq n \leq m \leq 100\,000$.

The input must be read from standard input.

Output

Print the minimum number of tries that the thief must perform in order to know what key opens each door with an end of line.

The output must be written to standard output.

Sample Input	Sample Output
3	14
4 6	1
1 2	324750
500 900	

E - Frequent Values

Source file name: frequent.c, frequent.cpp, frequent.java, or frequent.py

You are given a sequence of n integers $a_1, a_2, \dots, a_n$ in non-decreasing order. In addition to that, you are given several queries consisting of indices i and j ($1 \leq i \leq j \leq n$). For each query, determine the most frequent value among the integers $a_i, \dots, a_j$.

Input

The input consists of several test cases. Each test case starts with a line containing two integers n and q ($1 \leq n, q \leq 100\,000$). The next line contains n integers $a_1, \dots, a_n$ ($-100\,000 \leq a_i \leq 100\,000$, for each $i \in \{1, \dots, n\}$) separated by You can assume $a_i \leq a_{i+1}$ for each $i \in \{1, \dots, n-1\}$. The following q lines contain one query each, consisting of two integers i and j ($1 \leq i \leq j \leq n$), which indicate the boundary indices for the query.

The last case is followed by a line containing a single '0'.

The input must be read from standard input.

Output

For each query, print one line with one integer: the number of occurrences of the most frequent value within the given range.

The output must be written to standard output.

Sample Input	Sample Output
10 3	1
-1 -1 1 1 1 1 3 10 10 10	4
2 3	3
1 10	
5 10	
0	

F - Frosh Week

Source file name: `frosh.c`, `frosh.cpp`, `frosh.java`, or `frosh.py`

During Frosh Week, students play various fun games to get to know each other and compete against other teams. In one such game, all the frosh on a team stand in a line, and are then asked to arrange themselves according to some criterion, such as their height, their birth date, or their student number. This rearrangement of the line must be accomplished only by successively swapping pairs of consecutive students. The team that finishes fastest wins. Thus, in order to win, you would like to minimize the number of swaps required

Input

Input contains several test cases. For each test case, the first line of input contains one positive integer n , the number of students on the team, which will be no more than one million. The following n lines each contain one integer, the student number of each student on the team. No student number will appear more than once.

The input must be read from standard input.

Output

For each test case, output a line containing the minimum number of swaps required to arrange the students in increasing order by student number.

The output must be written to standard output.

Sample Input	Sample Output
3 3 1 2	2

G - Magic Numbers

Source file name: magic.c, magic.cpp, magic.java, or magic.py

Write a program that finds and displays all pairs of integers s_1 and s_2 such that:

1. neither s_1 nor s_2 have any digits repeated; and
2. $\frac{s_1}{s_2} = N$, where N is a given integer.

Input

The first line contains a single integer $T \geq 0$ indicating the number of test cases. Then T test cases follow. A test case consists of a single positive integer N .

There is a blank line before each test case.

The input must be read from standard input.

Output

For each test case output a sequence of zero or more lines each following the pattern

$$s_1/s_2 = N,$$

where s_1 , s_2 , and N are integers described above. If there is more than one triple per test case, list them in ascending order with respect to s_2 .

Print a blank line between the output of two consecutive test cases. See the sample output for details.

The output must be written to standard output.

Sample Input	Sample Output
2	1245698 / 1 = 1245698
1245698	12456980 / 10 = 1245698
52369874	73496182 / 59 = 1245698
	429765810 / 345 = 1245698
	734961820 / 590 = 1245698
	9215673804 / 7398 = 1245698
	52369874 / 1 = 52369874
	261849370 / 5 = 52369874
	523698740 / 10 = 52369874

H - Scrolling Sign

Source file name: `sign.c`, `sign.cpp`, `sign.java`, or `sign.py`

Electric scrolling signs are often used for advertising. A given sign displays exactly k characters. When the sign is switched on, all of the character positions are initially empty (showing spaces). In each time interval, all of the characters on the sign are shifted to the left by one position, and a new character is added at the right-most position. The character that was in the left-most position moves off the sign.

For certain sequences of words, it is possible to reuse characters from one word to form a subsequent word. For example, on a sign with three character positions, the sign can display the message CAT ATE TED by scrolling in the five characters CATED.

The advertiser has a specific message to show using the sign. The faster the message is displayed, the more people will be able to see the whole message. Therefore, your job is to find a way to display all the words of the message by scrolling in the smallest number of letters. In between showing the words of the message, the sign may display other words that are not considered part of the message. However, the words of the message must be shown in the order in which they are given. If a word appears two or more times consecutively in the message, the sign only needs to show it once.

Input

The first line of input contains a single integer n , the number of test cases to follow. Each test case starts with a line containing a two integers, k , the number of character positions on the sign, and w , the number of words in the message. Each of the two integers is between 1 and 1 500, inclusive. The following w lines each contain a word of the message comprising exactly k uppercase letters.

The input must be read from standard input.

Output

For each test case, output a line containing a single integer, the minimum number of letters that must be scrolled into the sign so that it displays all the words of the message.

The output must be written to standard output.

Sample Input	Sample Output
2	5
3 2	5
CAT	
TED	
3 3	
CAT	
ATE	
TEA	

I - Integer Sequences from Addition of Terms

Source file name: integer.c, integer.cpp, integer.java, or integer.py

We consider sequences formed from the addition of terms of a given sequence. Let $\{a_n\}$, $n = 1, 2, 3, \dots$ be an arbitrary sequence of integer numbers; d a positive integer. We construct another sequence $\{b_m\}$, $m = 1, 2, 3, \dots$ by defining b_m as consisting of $n \times d$ occurrences of the term a_n :

$$b_1 = \underbrace{a_1, \dots, a_1}_{d \text{ occurrences of } a_1}, b_2 = \underbrace{a_2, \dots, a_2}_{2d \text{ occurrences of } a_2}, b_3 = \underbrace{a_3, \dots, a_3}_{3d \text{ occurrences of } a_3}, \dots$$

For example, if $a_n = n$ and $d = 1$, then the resulting sequence $\{b_m\}$ is:

$$\underbrace{1}_{b_1}, \underbrace{2, 2}_{b_2}, \underbrace{3, 3, 3}_{b_3}, \underbrace{4, 4, 4, 4}_{b_4}, \dots$$

Given a_n and d we want to obtain the corresponding k -th digit integer in the sequence $\{b_m\}$. For example, with $a_n = n$ and $d = 1$ we have: 3 for $k = 6$ and 4 for $k = 7$. With $a_n = n$ and $d = 2$, we have: 2 for $k = 6$ and 3 for $k = 7$.

Input

The first line of input contains C ($C \geq 0$), the number of test cases that follow.

Each of the C test cases consists of three lines:

1. The first line represents a_n -- a polynomial in n of degree i with non-negative integer coefficients in increasing order of the power:

$$a_n = c_0 + c_1n + c_2n^2 + c_3n^3 + \dots + c_in^i$$

where $c_j \in \mathbb{N}$, $j = 0, \dots, i$. This polynomial a_n is codified by its degree i followed by the coefficients c_j , $j = 0, \dots, i$. All the numbers are separated by a single space.

2. The second line is the positive integer d .
3. The third line is the positive integer k .

It is assumed that the polynomial a_n is a polynomial of degree at most 20 ($1 \leq i \leq 20$) with non-negative integer coefficients at most 10 000 ($0 \leq c_j \leq 10\,000$, $j = 0, \dots, i$), $1 \leq d \leq 100\,000$, and $1 \leq k \leq 1\,000\,000$.

The input must be read from standard input.

Output

The output is a sequence of lines, one for each test case. Each of these lines contains the k -th integer in the sequence $\{b_m\}$ for the corresponding test case. This value is at most $2^{63} - 1$.

The output must be written to standard output.

Sample Input	Sample Output
2	1866
4 3 0 0 0 23	3
25	
100	
1 0 1	
1	
6	

J - The Mailbox Manufacturers Problem

Source file name: mailbox.c, mailbox.cpp, mailbox.java, or mailbox.py

In the good old days when Swedish children were still allowed to blow up their fingers with fire-crackers, gangs of excited kids would plague certain smaller cities during Easter time, with only one thing in mind: to blow things up. Small boxes were easy to blow up, and thus mailboxes became a popular target.

A small mailbox manufacturer is interested in how many fire-crackers his new mailbox prototype can withstand without exploding and has hired you to help him. He will provide you with k ($1 \leq k \leq 10$) identical mailbox prototypes each fitting up to m ($1 \leq m \leq 100$) crackers.

However, he is not sure of how many fire-crackers he needs to provide you with in order for you to be able to solve his problem, so he asks you. You think for a while and then say: “Well, if I blow up a mailbox I can’t use it again, so if you would provide me with only $k = 1$ mailboxes, I would have to start testing with 1 cracker, then 2 crackers, and so on until it finally exploded. In the worst case, that is if it does not blow up even when filled with m crackers, I would need $1 + 2 + 3 + \dots + m = \frac{m(m+1)}{2}$ crackers. If $m = 100$ that would mean more than 5000 fire-crackers!”. “That’s too many”, he replies. “What if I give you more than $k = 1$ mailboxes? Can you find a strategy that requires less crackers?”

Can you? And what is the minimum number of crackers that you should ask him to provide you with?

You may assume the following:

1. If a mailbox can withstand x fire-crackers, it can also withstand $x - 1$ firecrackers.
2. Upon an explosion, a mailbox is either totally destroyed (blown up) or unharmed, which means that it can be reused in another test explosion.

Note that if the mailbox can withstand a full load of m fire-crackers, then the manufacturer will of course be satisfied with that answer. But otherwise he is looking for the maximum number of crackers that his mailboxes can withstand.

Input

he input starts with a single integer N ($N \geq 0$) indicating the number of test cases to follow. Each test case is described by a line containing two integers: k and m , separated by a single space.

The input must be read from standard input.

Output

For each test case print one line with a single integer indicating the minimum number of fire-crackers that is needed, in the worst case, in order to figure out how many crackers the mailbox prototype can withstand.

The output must be written to standard output.

Sample Input	Sample Output
4	55
1 10	5050
1 100	382
3 73	495
5 100	

K - Asteroid Rangers

Source file name: `asteroid.c`, `asteroid.cpp`, `asteroid.java`, or `asteroid.py`

The year is 2112 and humankind has conquered the solar system. The Space Ranger Corps have set up bases on any hunk of rock that is even remotely inhabitable. Your job as a member of the Asteroid Communications Ministry is to make sure that all of the Space Ranger asteroid bases can communicate with one another as cheaply as possible. You could set up direct communication links from each base to every other base, but that would be prohibitively expensive. Instead, you want to set up the minimum number of links so that everyone can send messages to everyone else, potentially relayed by one or more bases. The cost of any link is directly proportional to the distance between the two bases it connects, so this doesn't seem that hard of a problem.

There is one small difficulty, however. Asteroids have a tendency to move about, so two bases that are currently very close may not be so in the future. Therefore as time goes on, you must be willing to switch your communication links so that you always have the cheapest relay system in place. Switching these links takes time and money, so you are interested in knowing how many times you will have to perform such a switch.

A few assumptions make your task easier. Each asteroid is considered a single point. Asteroids always move linearly with a fixed velocity. No asteroids ever collide with other asteroids. Also, any relay system that becomes optimal at a time $t \geq 0$ will be uniquely optimal for any time s satisfying $t < s < t + 10^{-6}$.

Input

Each test case starts with a line containing an integer n ($2 \leq n \leq 50$) indicating the number of asteroid bases. Following this are n lines, each containing six integers x, y, z, v_x, v_y, v_z . The first three specify the initial location of an asteroid ($-150 \leq x, y, z \leq 150$), and the last three specify the x, y , and z components of that asteroid's velocity in space units per time unit ($-100 \leq v_x, v_y, v_z \leq 100$).

The input must be read from standard input.

Output

For each test case, display a single line containing the case number and the number of times that the relay system needs to be set up or modified. See the sample output for more details.

The output must be written to standard output.

Sample Input	Sample Output
<pre> 3 0 0 0 0 0 0 5 0 0 0 0 0 10 1 0 -1 0 0 4 0 0 0 1 0 0 0 1 0 0 -1 0 1 1 1 3 1 1 -1 -1 2 1 -1 -1 </pre>	<pre> Case 1: 3 Case 2: 3 </pre>